Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
MoE, UGC & AICTE Approved; NAAC Accredited A++
Karunya Nagar, Coimbatore - 641 114, Tamil Nadu, India.

DIVISION OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

PROJECT-BASED EVALUATION REPORT

Academic Year 2026-2027

ODD SEMESTER

Name : **ASWIN SAMUEL.A**

Register Number : **URK24CS9014**

Course Code : **23CS2024**

Course Name : **Data Science Ecosystem**

Oct 2026

RAILSAATHI AI

AI-Powered Complaint Management for Rail Madad (SIH1711)

ASWIN SAMUEL.A (URK24CS9014)

Smart India Hackathon 2024 · Problem Statement SIH1711 · Ministry of Railways

ABSTRACT

Rail Madad is the grievance platform through which railway passengers report problems during a journey. Complaints arrive as typed text, Hinglish messages, photographs, videos and voice notes, and today they are largely categorised, prioritised and forwarded by hand. RailSaathi AI is a data-science pipeline, developed for Smart India Hackathon problem statement SIH1711, that automates these steps.

A process-driven simulator produced 25,133 realistic multimodal complaints for 60 trains. Complaint text, OCR text from photos and image tags are fused into one TF-IDF feature space. A calibrated linear SVM assigns the category with 95.0% accuracy, and confidence-aware routing automates 97.6% of assignments at 97.0% accuracy. A cost-sensitive urgency rule raises critical recall to 94.9%. In a queue simulation, it cuts the average wait for critical complaints from 535 to 32 minutes. Graph-based grouping merges duplicate reports, and Poisson gradient boosting flags deteriorating trains for predictive maintenance.

The trained model also runs inside an interactive WebGL website with a 3D complaint galaxy and a live demo. The project shows that lightweight, explainable machine learning can make complaint handling faster and safer.

TABLE OF CONTENTS

1. INTRODUCTION 4
BACKGROUND OF THE PROJECT 4
PROBLEM STATEMENT 4
OBJECTIVES 4
SCOPE OF THE PROJECT 4
2. LITERATURE REVIEW 4
REVIEW OF EXISTING RESEARCH 4
RESEARCH GAP AND LIMITATIONS OF EXISTING SYSTEMS 5
3. METHODOLOGY AND SYSTEM DESIGN 5
PROPOSED METHODOLOGY AND SYSTEM ARCHITECTURE 5
DATASET 5
TOOLS AND TECHNOLOGIES USED 6
MODULE DESCRIPTION 6
4. IMPLEMENTATION AND RESULTS 6
IMPLEMENTATION DETAILS 6
APPLICATION INTEGRATION AND DEPLOYMENT 6
RESULTS AND ANALYSIS 7
5. CONCLUSION AND FUTURE ENHANCEMENT 8
SUMMARY OF THE WORK 8
FUTURE ENHANCEMENTS 8
CONCLUSION 8
REFERENCES 9

ARISE AND SHINE
Karunya

1. INTRODUCTION

BACKGROUND OF THE PROJECT

Indian Railways carries millions of passengers every day, and Rail Madad is their single window for help and grievances [1]. A complaint may be about a broken AC, a dirty toilet, a rude staff member or a medical emergency, and it may arrive through the app, the web portal, the 139 helpline or social media. Each one has to be read, given a category, sent to the right department and handled in time. As volumes grow, doing this manually becomes slow and error-prone.

PROBLEM STATEMENT

Problem statement SIH1711 [2] asks for an AI solution that categorises and prioritises Rail Madad complaints, extracts extra information from photos, videos and metadata, routes complaints to the right official, predicts recurring issues for proactive maintenance and monitors performance through feedback sentiment. The difficulty is that complaints are short, noisy, often code-mixed (Hinglish) and sometimes consist of only an image or a voice note.

OBJECTIVES

1. Build a realistic multimodal complaint dataset with categories, severities and operational patterns.

2. Categorise complaints by fusing text, OCR and image tags, and route only confident predictions automatically.

3. Predict urgency with a cost-sensitive rule that avoids under-prioritising emergencies.

4. Extract train, coach and PNR details, merge duplicate reports and forecast maintenance hotspots.

5. Present the working model through an interactive web interface.

SCOPE OF THE PROJECT

The project covers data generation, preprocessing, machine-learning models, evaluation and an interactive website. It uses classical, explainable machine learning rather than large language models. Image and speech recognition are represented by their outputs (OCR text, image tags, transcripts), and the real Rail Madad platform is not modified.

2. LITERATURE REVIEW

REVIEW OF EXISTING RESEARCH

TF-IDF weighting [3] with linear classifiers remains a strong approach for short texts, and support vector machines handle the sparse, high-dimensional features of text well [4]. Platt scaling converts SVM scores into calibrated probabilities [5], and selective classification lets a model abstain when unsure [6]. When errors have unequal costs, choosing the class with minimum expected cost is optimal [7]. Queueing theory shows why priority ordering shortens waits for

urgent jobs [8]. Poisson models suit count data [9], and t-SNE is a standard tool for visualising high-dimensional data in two or three dimensions [10]. Hindi-English code-mixing is common in Indian social media text and complicates word-based models [11].

RESEARCH GAP AND LIMITATIONS OF EXISTING SYSTEMS

The existing Rail Madad workflow relies on manual or menu-based categorisation, handles complaints largely first-come-first-served, does not mine the text and metadata inside attached media, registers duplicate reports of one fault as separate tickets and reacts to faults instead of predicting them. Most published complaint classifiers also optimise accuracy alone, ignoring the much higher cost of missing an emergency.

3. METHODOLOGY AND SYSTEM DESIGN

PROPOSED METHODOLOGY AND SYSTEM ARCHITECTURE

RailSaathi AI converts every complaint into one fused document: typed text or a voice transcript, OCR text from photos (prefixed ocr_) and image-model tags (prefixed img_). Hinglish and SMS words are normalised with a small lexicon, and word 1-2-gram and character n-gram TF-IDF features are extracted. Three models then act on this representation (Fig. 1): categorisation with confidence-aware routing, cost-sensitive urgency detection and duplicate grouping. An analytics layer adds maintenance forecasting, SLA-breach prediction and sentiment monitoring.

RailSaathi AI: proposed AI-assisted Rail Madad complaint pipeline

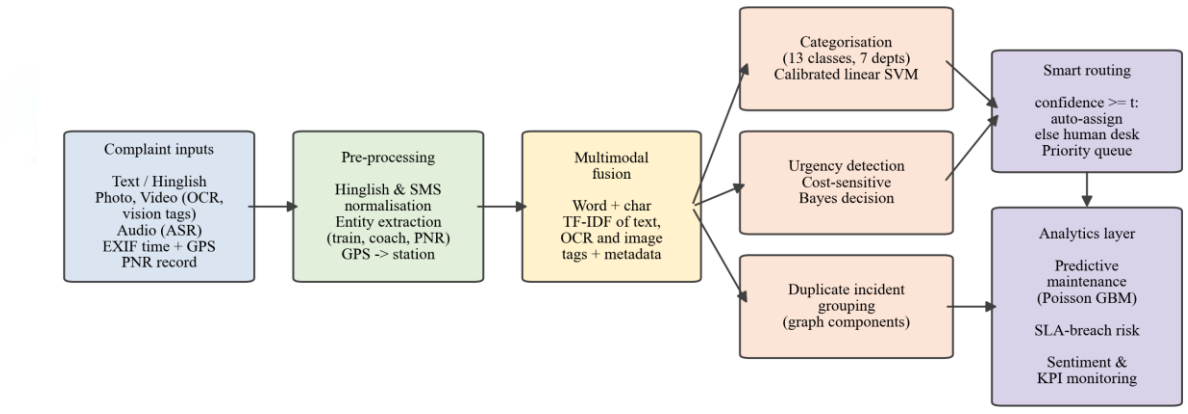


Fig. 1. System architecture of RailSaathi AI.

DATASET

Because real Rail Madad records are not public, a simulator generated 25,133 complaints across 13 categories, 7 departments and 4 severity levels (January 2025 to September 2026). Each train has a latent condition that evolves over time, eight trains deteriorate steadily, and seasonal effects such as summer AC failures and winter fog delays are built in. 18% of complaints are in Hinglish,

many contain typos, 4% are vague and 3% carry wrong labels, which keeps the task realistic. The last six months form the test set.

TOOLS AND TECHNOLOGIES USED

Python 3.11 with pandas, NumPy and SciPy was used for data handling. scikit-learn [12] provided the models, Matplotlib the figures and python-docx this report. The website uses HTML, CSS, JavaScript and Three.js (WebGL) [13], and Git and GitHub were used for version control and hosting.

MODULE DESCRIPTION

Categorisation and routing: compares Naive Bayes, logistic regression, calibrated linear SVM and random forest, and auto-routes a complaint only when its confidence exceeds a threshold.

Urgency detection: applies a cost matrix in which predicting Critical as Low costs 16 and over-triage costs 1, and picks the severity with minimum expected cost.

Extraction and grouping: uses regular expressions for train, coach and PNR; GPS for the nearest station; and links reports of the same train, coach, category and 6-hour window into one incident.

Predictive maintenance: uses Poisson gradient boosting on weekly complaint counts per train.

Feedback analytics: includes a sentiment classifier and an SLA-breach risk model.

4. IMPLEMENTATION AND RESULTS

IMPLEMENTATION DETAILS

Each module is a separate Python script, and run_all.py executes the complete pipeline in under two minutes on a laptop. All randomness is seeded, so every reported number is reproducible. Results are saved as CSV/JSON files and figures as 300-dpi images. The source code is available at https://github.com/aswinsamuel-codes/RailSaathi-AI.

APPLICATION INTEGRATION AND DEPLOYMENT

A compact version of the model (word and character TF-IDF with logistic regression) was exported to JavaScript, so the website classifies complaints entirely in the browser. On 300 test complaints it produced exactly the same predictions as Python, with 94.0% accuracy. The website (Fig. 2) shows 2,600 test complaints as a rotatable 3D t-SNE galaxy and offers a live demo, animated result charts and the report figures. It is hosted with GitHub Pages from the docs folder.

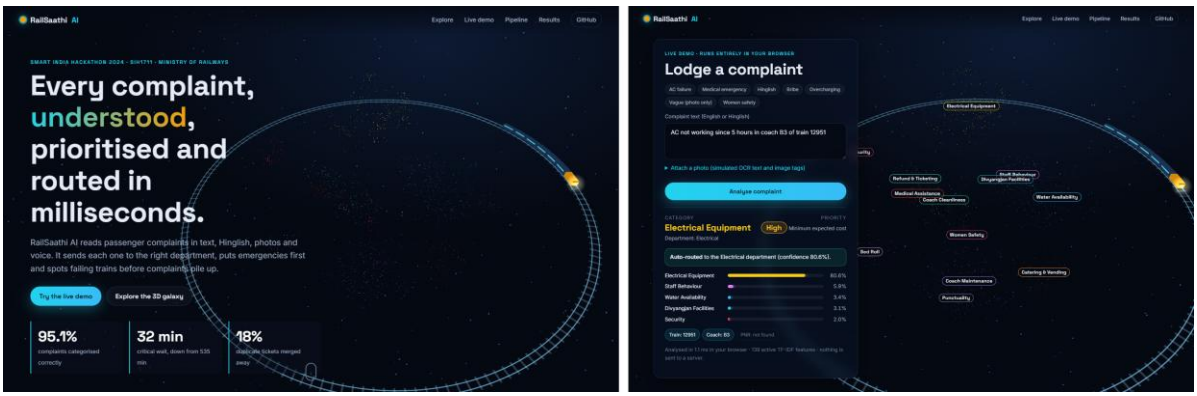


Fig. 2. RailSaathi AI website: 3D complaint galaxy (left) and live demo (right).

RESULTS AND ANALYSIS

Table 1. Categorisation results on the six-month test set (* best).

Model	Accuracy (%)	Macro-F1 (%)	Correct department (%)
Complement Naive Bayes	93.17	90.63	94.23
Logistic Regression	93.81	90.35	94.55
Linear SVM (calibrated)*	95.05	93.42	95.93
Random Forest	92.9	90.38	93.94

The calibrated linear SVM performed best (Table 1). Adding OCR text and image tags raised accuracy on photo/video complaints from 93.7% to 96.5%. For urgency, plain arg-max logistic regression missed 9.1% of critical cases. The cost-sensitive rule lowered under-triage from 6.4% to 1.6% at a small cost in overall accuracy, and in the queue simulation critical complaints were served almost immediately (Fig. 3).

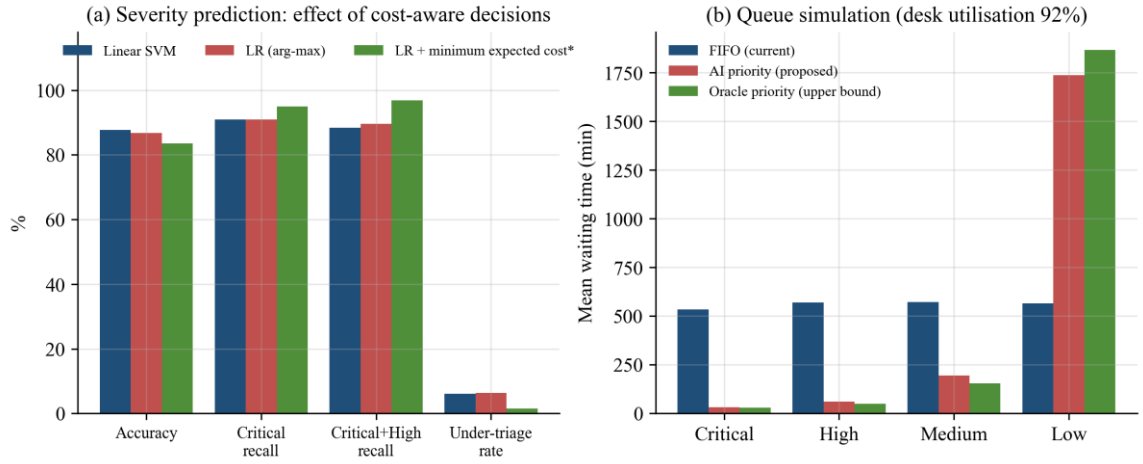


Fig. 3. (a) Effect of the cost-sensitive rule; (b) mean waiting time by severity under three queueing policies.

For predictive maintenance, Poisson gradient boosting reduced forecast deviance by 65% compared with a naive last-week forecast. Ranking trains by predicted complaints flagged deteriorating rakes at 50% precision, against 13% by chance (Fig. 4). Table 2 summarises all modules.

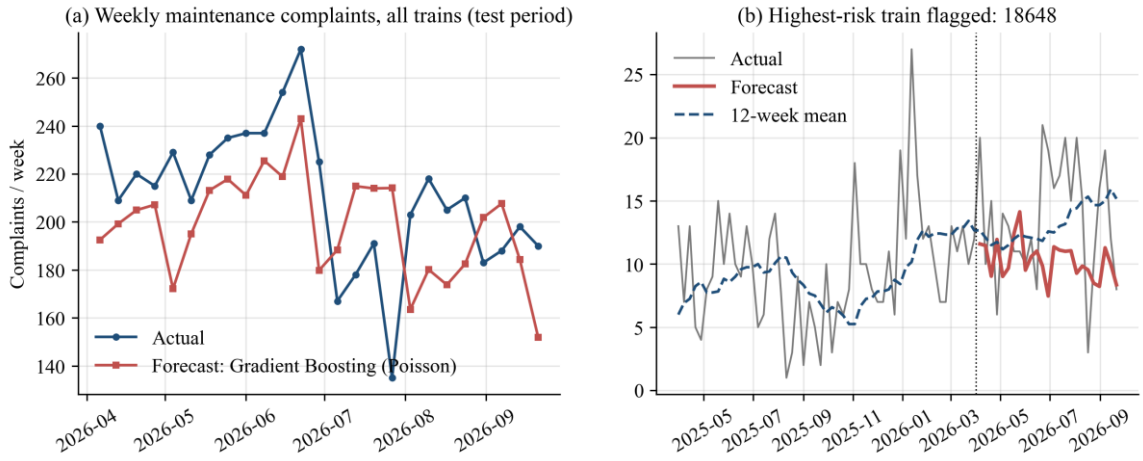


Fig. 4. (a) Weekly maintenance-complaint forecast for all trains; (b) history and forecast of the highest-risk train.

Table 2. Summary of results of all modules.

Module	Key result
Smart routing	97.6% auto-routed at 97.0% accuracy
Urgency (critical recall)	90.9% to 94.9%
Critical waiting time	535 min to 32 min
Entity extraction (OCR)	Train no. recovered 41% to 51%; GPS station 99.8%
Duplicate grouping	Pair F1 0.97; 8,056 to 6,630 tickets
Predictive maintenance	Deviance -65% vs naive; 50% of flagged trains truly failing (13% by chance)
Feedback analytics	Sentiment accuracy 80.8%; SLA-breach ROC-AUC 0.73

5. CONCLUSION AND FUTURE ENHANCEMENT

SUMMARY OF THE WORK

RailSaathi AI turns raw, multimodal Rail Madad complaints into categorised, prioritised and de-duplicated tickets, and turns the complaint stream into maintenance and service-quality insights. Multimodal fusion helps most where manual handling struggles, on photo and video complaints. Cost-aware decisions protect emergencies, and metadata such as the booked coach makes duplicate detection reliable.

FUTURE ENHANCEMENTS

Future work includes validating the system on real, anonymised Rail Madad data. Real image and speech models could replace the simulated tags and transcripts, and multilingual transformer embeddings could cover regional languages. A chatbot could ask passengers for missing PNR or coach details. Maintenance alerts could also be linked to depot schedules for staff allocation.

CONCLUSION

The project shows that explainable machine learning, evaluated with the right metrics, can make railway grievance handling faster, fairer and more proactive. The main limitation is the synthetic dataset, so absolute scores may be lower on real records.

REFERENCES

[1] Ministry of Railways, Government of India, "Rail Madad." [Online]. Available: https://railmadad.indianrailways.gov.in

[2] Ministry of Education's Innovation Cell, "Smart India Hackathon 2024, Problem Statement SIH1711: Enhancing Rail Madad with AI-powered complaint management," 2024.

[3] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Inf. Process. Manage., vol. 24, no. 5, pp. 513-523, 1988.

[4] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in Proc. ECML, 1998, pp. 137-142.

[5] J. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in Advances in Large Margin Classifiers, MIT Press, 1999, pp. 61-74.

[6] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in Proc. NeurIPS, 2017.

[7] C. Elkan, "The foundations of cost-sensitive learning," in Proc. IJCAI, 2001, pp. 973-978.

[8] L. Kleinrock, Queueing Systems, Vol. 1: Theory. New York, NY, USA: Wiley, 1975.

[9] P. McCullagh and J. A. Nelder, Generalized Linear Models, 2nd ed. London, U.K.: Chapman & Hall, 1989.

[10] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," J. Mach. Learn. Res., vol. 9, pp. 2579-2605, 2008.

[11] K. Bali, J. Sharma, M. Choudhury, and Y. Vyas, "'I am borrowing ya mixing?' An analysis of English-Hindi code mixing in Facebook," in Proc. 1st Workshop Comput. Approaches Code Switching, 2014, pp. 116-126.

[12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[13] three.js authors, "three.js - JavaScript 3D library." [Online]. Available: https://threejs.org

ARISE AND SHINE
Karunya

EVALUATION SHEET

Reg.No : **URK24CS9014**

Name : **ASWIN SAMUEL.A**

Title of the Project : **RAILSAATHI AI**

Course Code : **23CS2024**

Course Name : **Data Science Ecosystem**

S.No	Rubrics	Maximum Marks	Marks Obtained
1	Evaluation of Problem Statement, Methodology, and Dataset	5	
2	Implementation and Result Analysis	5	
3	Demonstration and Viva	5	
4	Report	5	
5	Microsoft Certificate	20	
Total		40	

Signature of the Faculty-in-charge